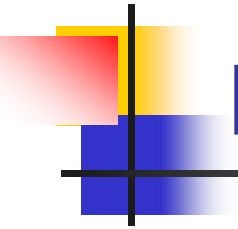
File Management

Chapter 12

File Management System

- File management system consists of system utility programs that run as privileged applications
- Input to applications is by means of a file
- Output is saved in a file for long-term storage
- Sharable between processes



File Operations

- Create
- Delete
- Open
- Close
- Read
- Write

Objectives for a File System

- Meet the data management needs and requirements of the user
- Guarantee that the data in the file are valid
- Provide I/O support for a variety of storage device types
- Minimize or eliminate the potential for lost or destroyed data
- Provide a standardized set of I/O interface routines
- Provide I/O support for multiple users

FS Software Architecture

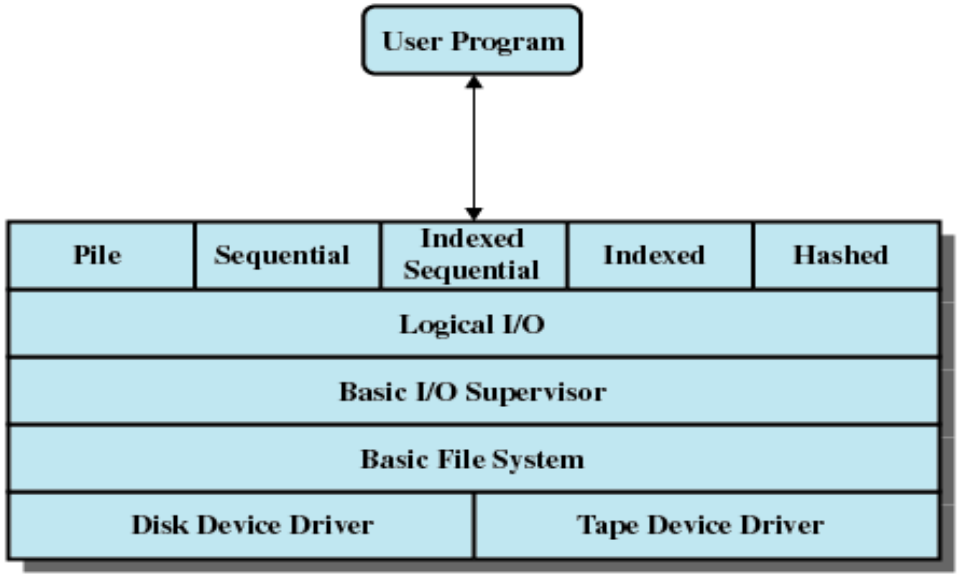
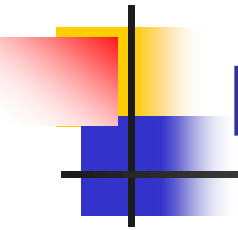


Figure 12.1 File System Software Architecture



Device Drivers

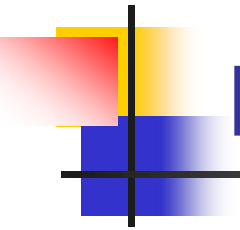
- Communicates directly with peripheral devices
- Responsible for starting I/O operations on a device
- Processes the completion of an I/O request

Basic File System

- Physical I/O
- Deals with exchanging blocks of data
- Concerned with the placement of blocks
- Concerned with buffering blocks in main memory

Basic I/O Supervisor

- Responsible for file I/O initiation and termination
- Concerned with selection of the device on which file I/O is to be performed
- Concerned with scheduling access to optimize performance



Logical I/O

- Enables users and applications to access file as a byte stream
- Maintains basic data about file

Criteria for File Organization

- Short access time
- Ease of update
  - File on CD-ROM will not be updated, so this is not a concern
- Economy of storage
  - Should be minimum redundancy in the data
  - Redundancy can be used to speed access such as an index
- Simple maintenance
- Reliability

File Directories

- Contains information about files
  - Attributes(属性)
  - Location(位置)
  - Ownership(所有者关系)
- Directory itself is a file owned by the operating system
- Provides mapping between file names and the files themselves

Tree-Structured Directory

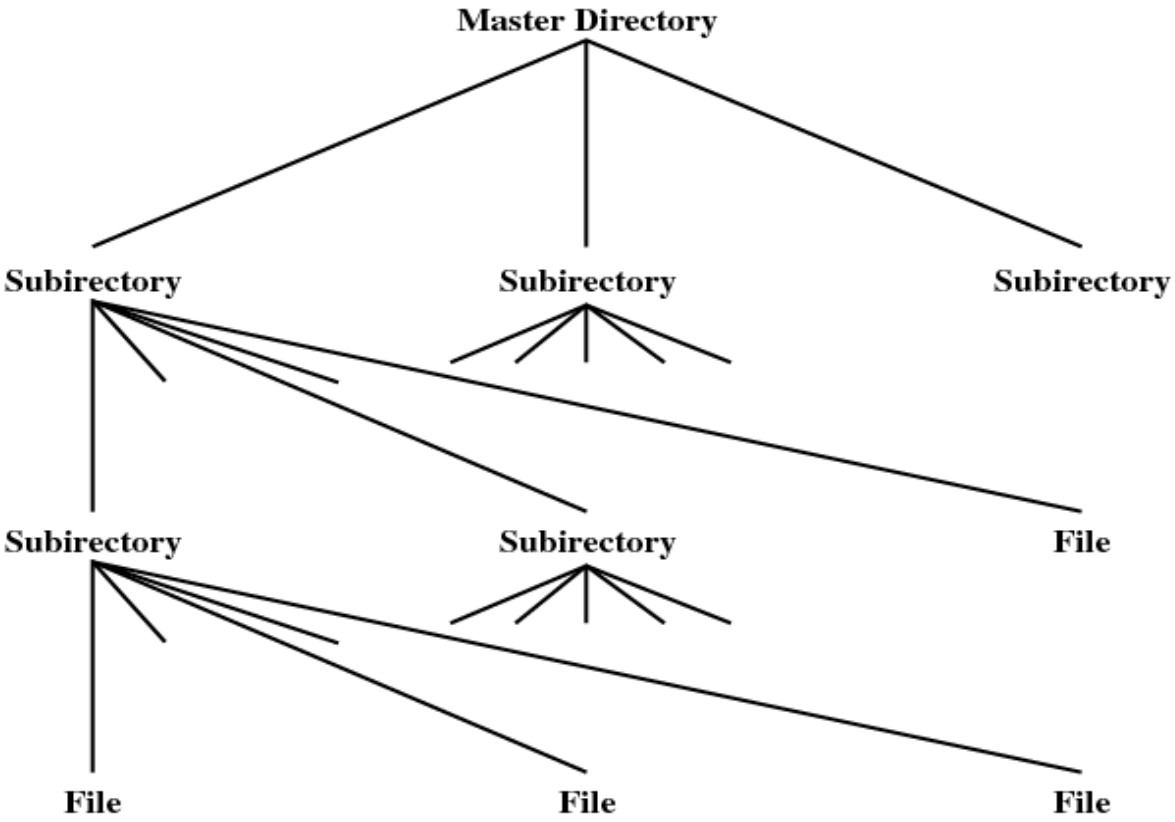


Figure 12.4 Tree-Structured Directory

Hierarchical, or Tree-Structured Directory

- Files can be located by following a path from the root, or master directory down various branches
  - This is the pathname for the file
- Can have several files with the same file name as long as they have unique path names
- Current directory is the working directory
- Files are referenced relative to the working directory

File Sharing

- In multiuser system, allow files to be shared among users
- Two issues
  - Access rights(存取/访问权限)
  - Management of simultaneous access

Access Rights(存取/访问权限)

- None
  - User may not know of the existence of the file
  - User is not allowed to read the user directory that includes the file
- Knowledge
  - User can only determine that the file exists and who its owner is

Access Rights (cont.)

- Execution
  - The user can load and execute a program but cannot copy it
- Reading
  - The user can read the file for any purpose, including copying and execution
- Appending
  - The user can add data to the file but cannot modify or delete any of the file's contents

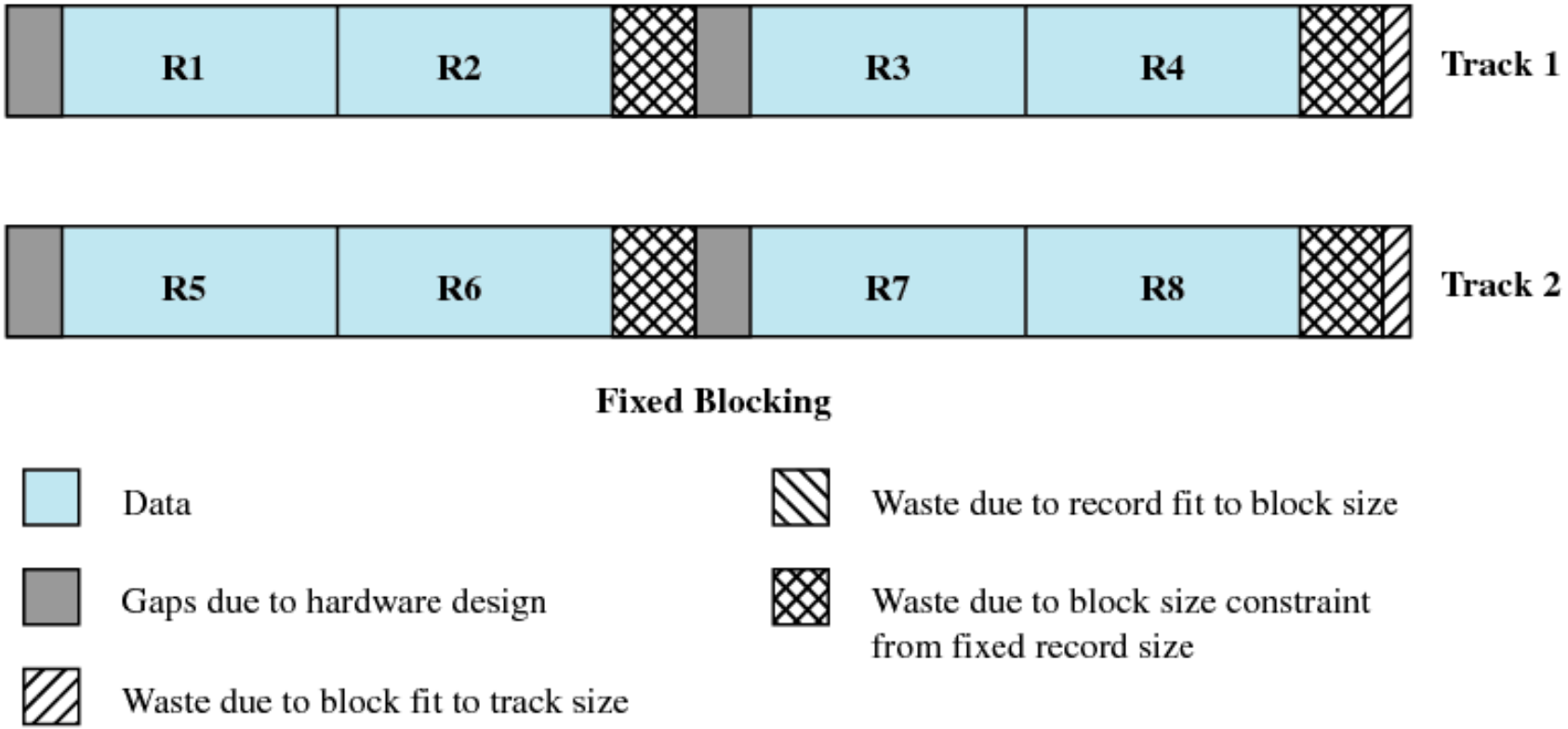
Access Rights (cont.)

- Updating
  - The user can modify, deleted, and add to the file's data. This includes creating the file, rewriting it, and removing all or part of the data
- Changing protection
  - User can change access rights granted to other users
- Deletion
  - User can delete the file

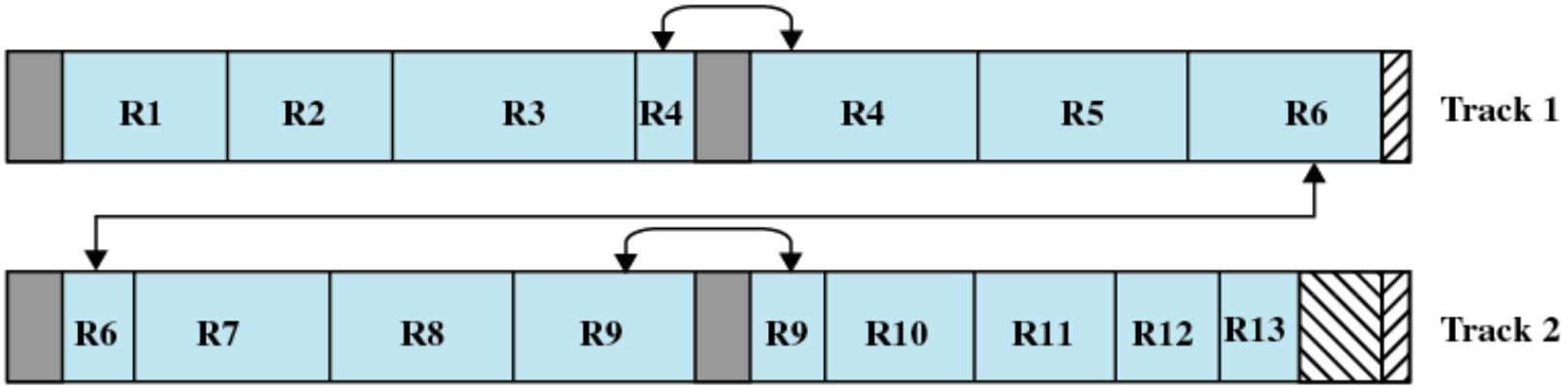
Access Rights (cont.)

- Owners
  - Has all rights previously listed
  - May grant rights to others using the following classes of users
    - Specific user
    - User groups
    - All for public files

Fixed Blocking



Variable Blocking: Spanned



Variable Blocking: Spanned

Data

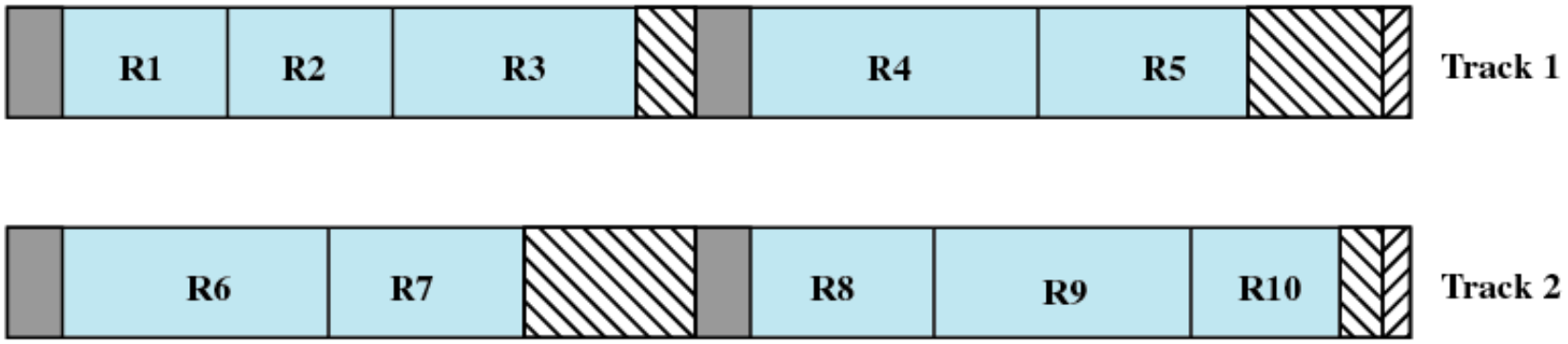
Gaps due to hardware design

Waste due to block fit to track size

Waste due to record fit to block size

Waste due to block size constraint from fixed record size

Variable Blocking: Unspanned



Variable Blocking: Unspanned

Data

Gaps due to hardware design

Waste due to block fit to track size

Waste due to record fit to block size

Waste due to block size constraint from fixed record size

Secondary Storage Management

- Space must be allocated to files
  - 辅助存储空间分配给了哪些文件
- Must keep track of the space available for allocation
  - 跟踪空闲空间，即记录哪些空闲空间是可用的

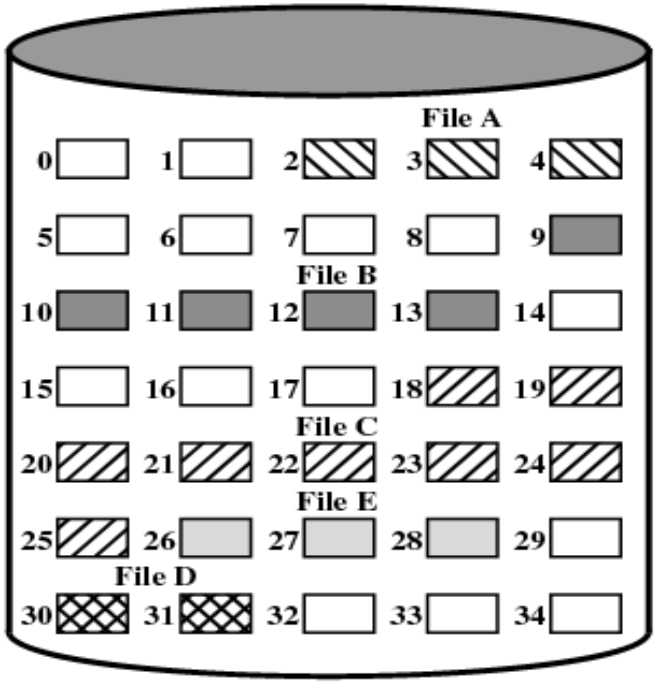
Preallocation(预先分配)

- Need the maximum size for the file at the time of creation
- Difficult to reliably estimate the maximum potential size of the file
- Tend to overestimated file size so as not to run out of space

Contiguous Allocation(连续分配)

- Single set of blocks is allocated to a file at the time of creation
- Only a single entry in the file allocation table(FAT, 文件分配表)
  - Starting block and length of the file
- External fragmentation will occur
  - Need to perform compaction

Contiguous Allocation

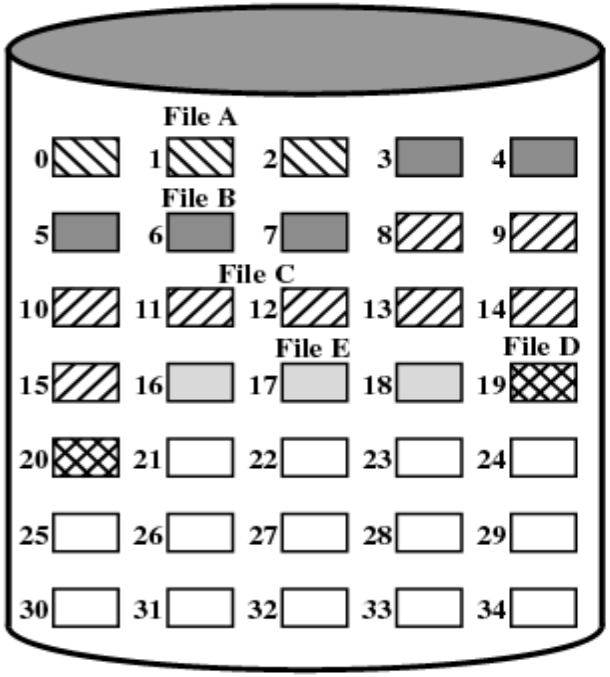


File Allocation Table

File Name	Start Block	Length
File A	2	3
File B	9	5
File C	18	8
File D	30	2
File E	26	3

Figure 12.7 Contiguous File Allocation

Contiguous Allocation (cont.)



File Allocation Table

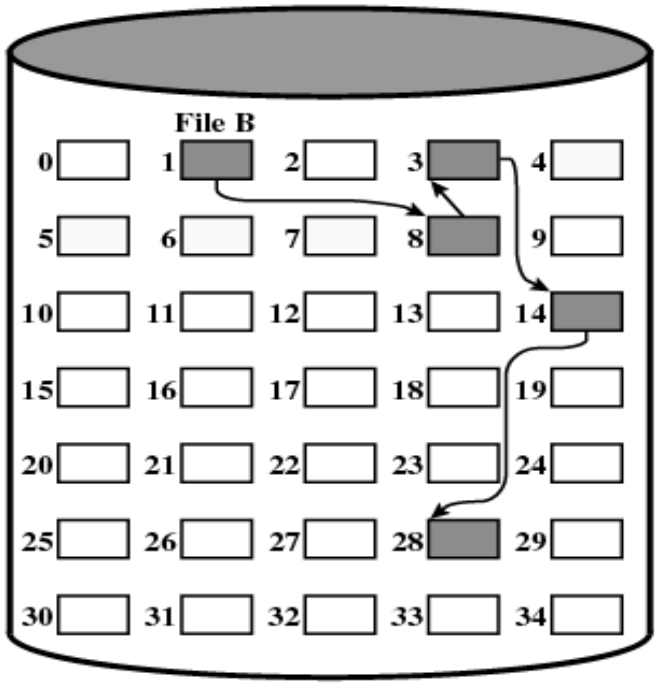
File Name	Start Block	Length
File A	0	3
File B	3	5
File C	8	8
File D	19	2
File E	16	3

Figure 12.8 Contiguous File Allocation (After Compaction)

Chained Allocation(非连续分配)

- Allocation on basis of individual block
- Each block contains a pointer to the next block in the chain
- Only single entry in the file allocation table
  - Starting block and length of file
- No external fragmentation
- No accommodation of the principle of locality

Chained Allocation (cont.)

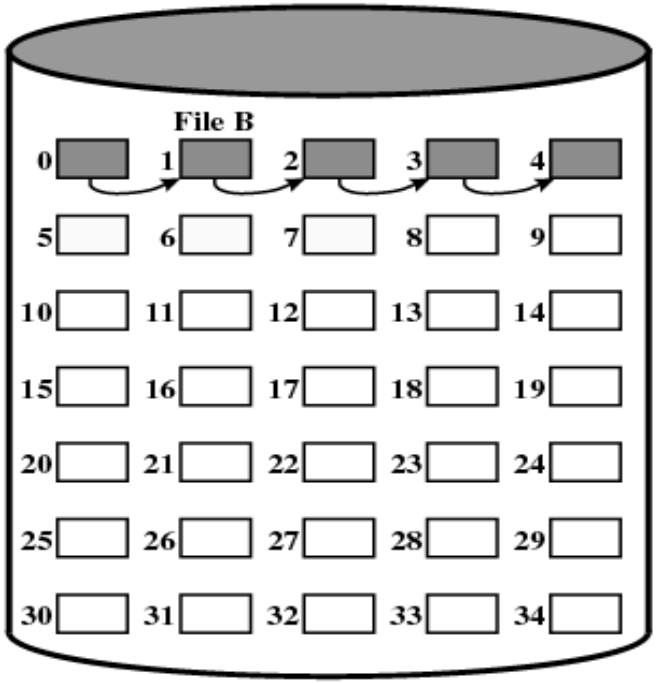


File Allocation Table

File Name	Start Block	Length
...	...	...
File B	1	5
...	...	...

Figure 12.9 Chained Allocation

Chained Allocation (cont.)



File Allocation Table

File Name	Start Block	Length
...	...	...
File B	0	5
...	...	...

Figure 12.10 Chained Allocation (After Consolidation)

Indexed Allocation (非连续分配)

- File allocation table contains a separate one-level index for each file
- The index has one entry for each portion allocated to the file
- The file allocation table contains block number for the index

Indexed Allocation (cont.)

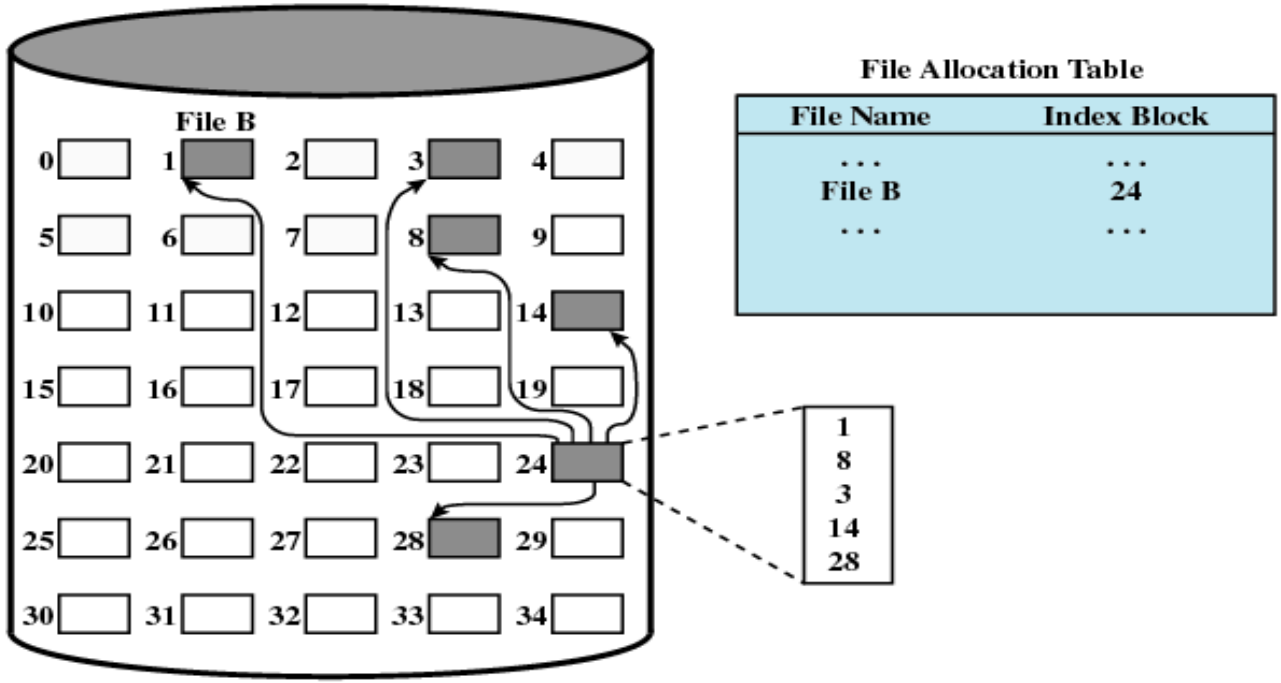


Figure 12.11 Indexed Allocation with Block Portions

Indexed Allocation (cont.)

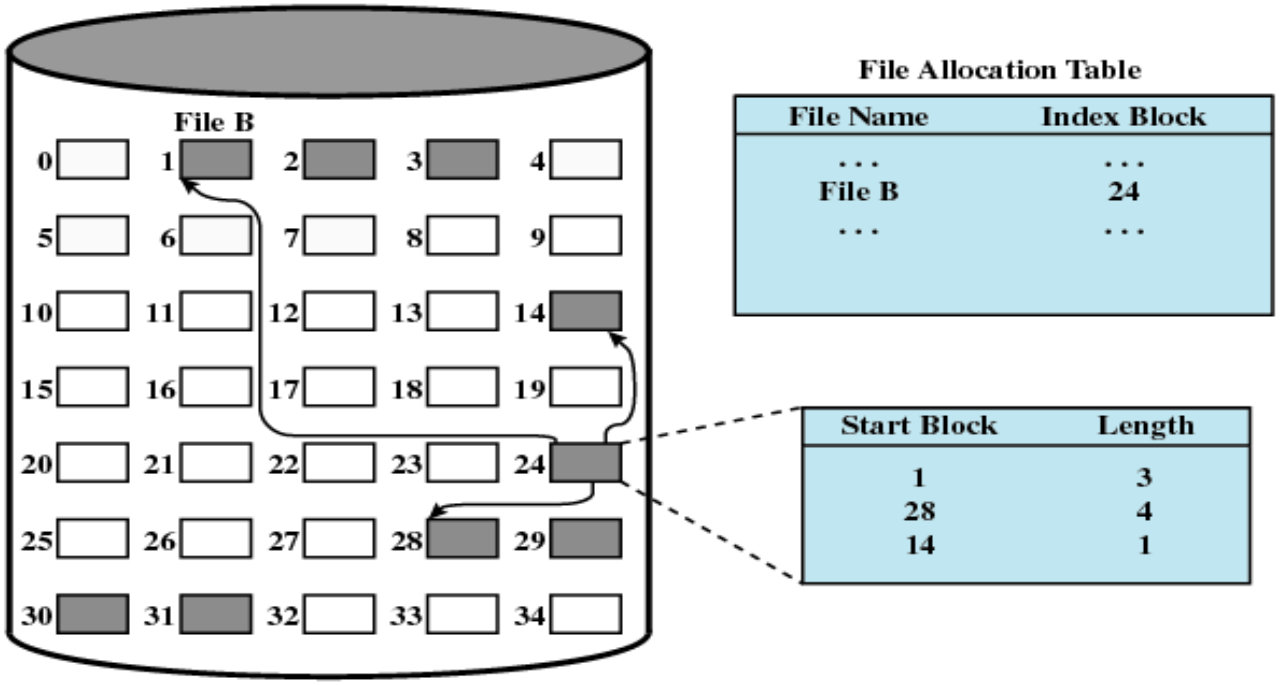
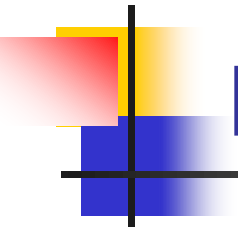


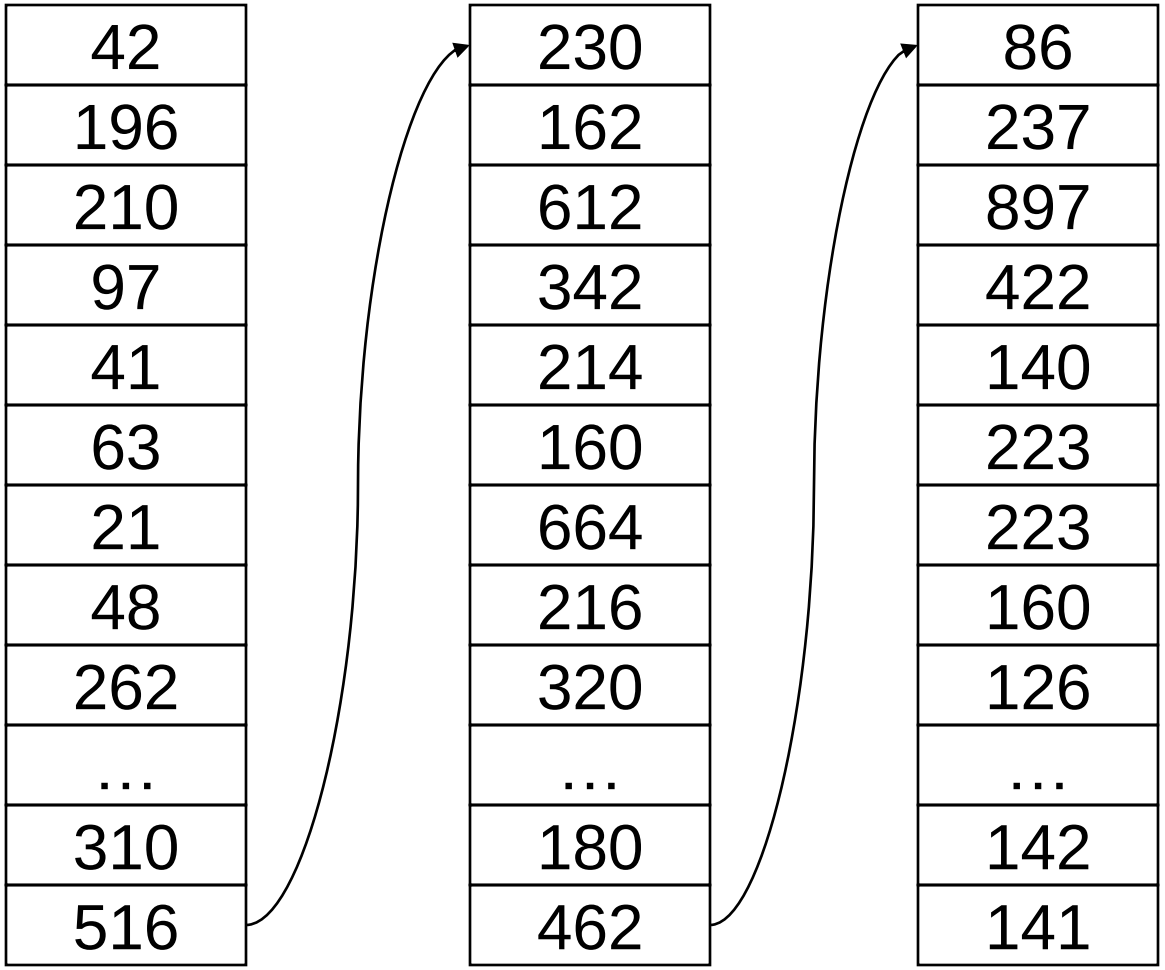
Figure 12.12 Indexed Allocation with Variable-Length Portions

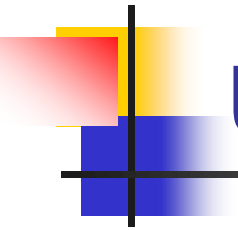


Free Space Management

- Bitmap
- Chained (Linked List)

Free Space Management





UNIX File Management

- Types of files
  - Regular, or ordinary
  - Directory
  - Special
  - Named pipes
  - Links
  - Symbolic links

Index node/I-node

- Index node (索引节点)
- Control structure that contains key information for a particular file

File Attributes of Unix File

Table 12.4 Information in a UNIX Disk-Resident Inode

File Mode	16-bit flag that stores access and execution permissions associated with the file. 12-14 File type (regular, directory, character or block special, FIFO pipe 9-11 Execution flags 8 Owner read permission 7 Owner write permission 6 Owner execute permission 5 Group read permission 4 Group write permission 3 Group execute permission 2 Other read permission 1 Other write permission 0 Other execute permission
Link Count	Number of directory references to this inode
Owner ID	Individual owner of file
Group ID	Group owner associated with this file
File Size	Number of bytes in file
File Addresses	39 bytes of address information
Last Accessed	Time of last file access
Last Modified	Time of last file modification
Inode Modified	Time of last inode modification

Layout of Unix File on Disk

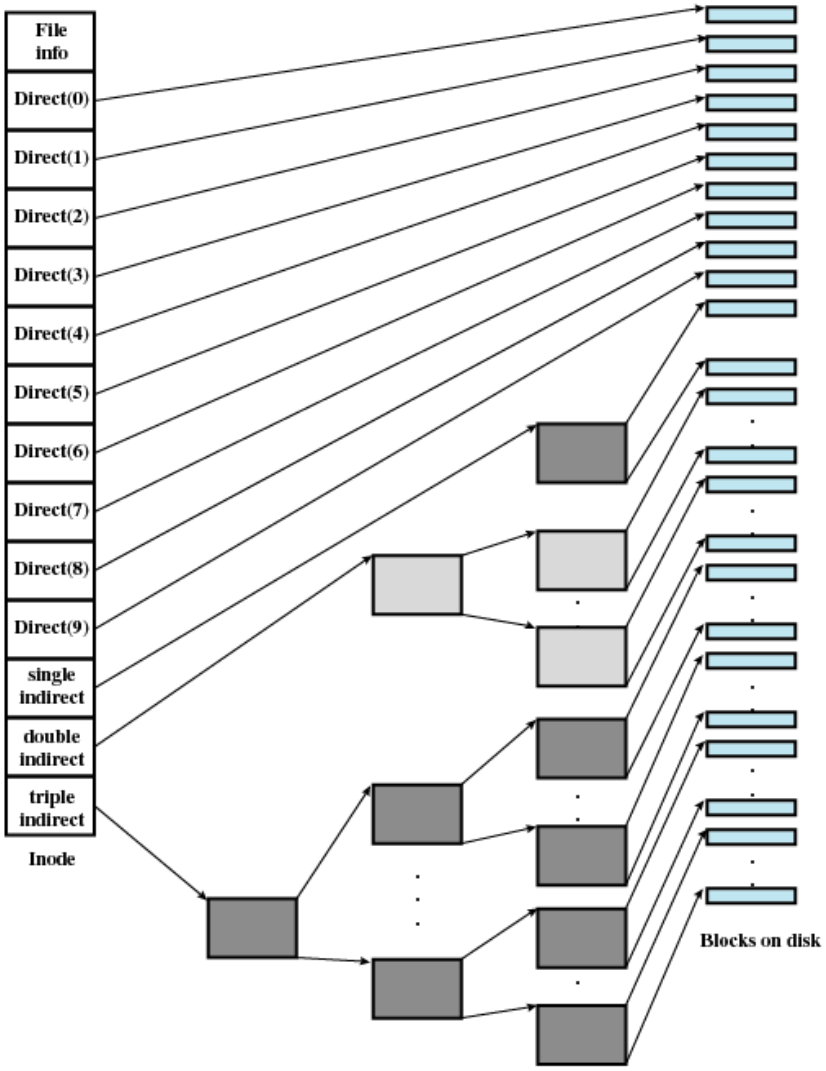
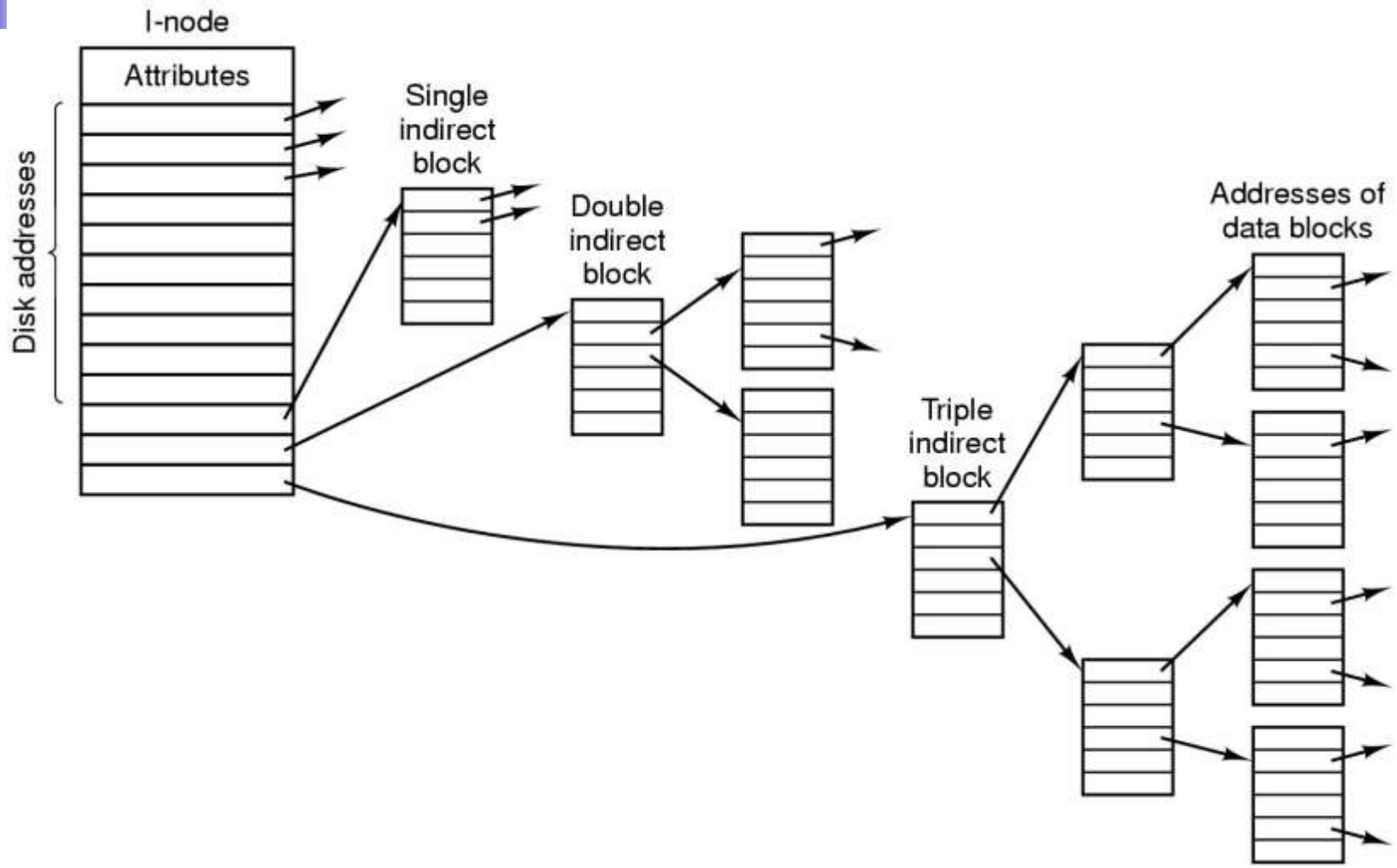


Figure 12.13 Layout of a UNIX File on Disk

Layout of Unix File on Disk



Unix Directory and I-node

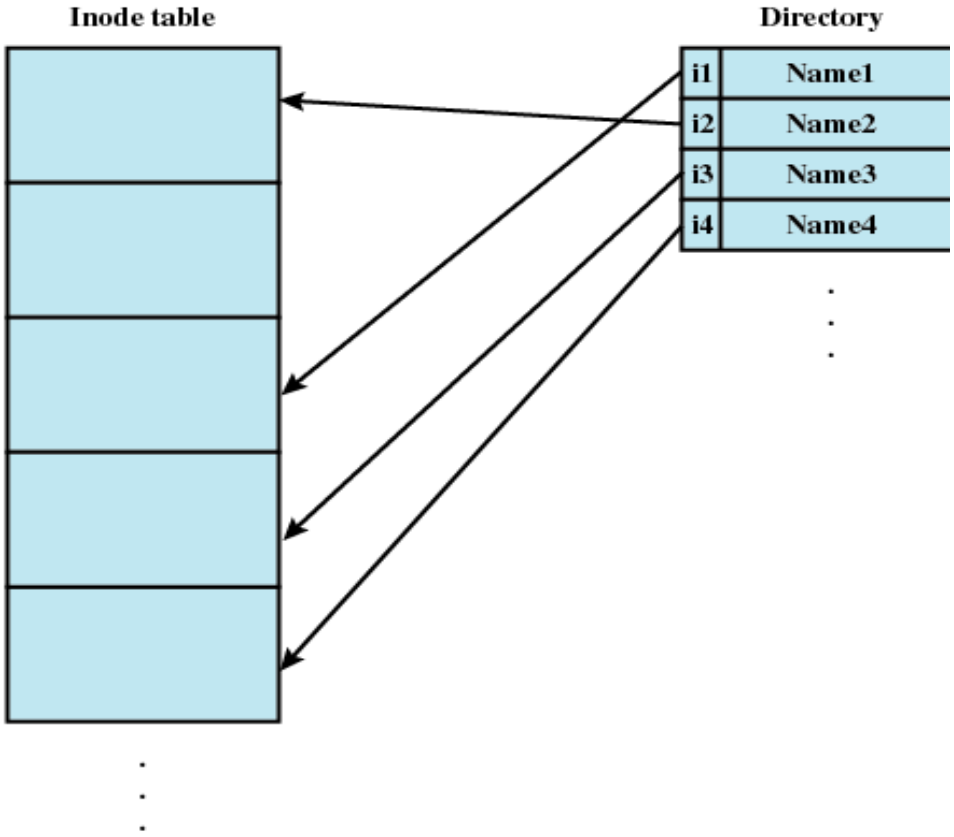


Figure 12.14 UNIX Directories and Inodes

Unix Pathname Parsing

- /usr/ast/mbox

Root dir

1	.
1	..
4	bin
7	dev
14	lib
9	etc
6	usr
8	tmp

i-node 6

Attirb
132

Block 132

6	.
1	..
19	dick
30	erik
51	jim
26	ast
45	bal

i-node 26

Attrib
406

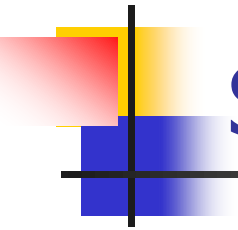
Block 406

26	.
6	..
64	grants
92	books
60	mbox
81	minix
17	src

Access Rights(Unix)

Table 12.4 Information in a UNIX Disk-Resident Inode

File Mode	16-bit flag that stores access and execution permissions associated with the file. 12-14 File type (regular, directory, character or block special, FIFO pipe 9-11 Execution flags 8 Owner read permission 7 Owner write permission 6 Owner execute permission 5 Group read permission 4 Group write permission 3 Group execute permission 2 Other read permission 1 Other write permission 0 Other execute permission
Link Count	Number of directory references to this inode
Owner ID	Individual owner of file
Group ID	Group owner associated with this file
File Size	Number of bytes in file
File Addresses	39 bytes of address information
Last Accessed	Time of last file access
Last Modified	Time of last file modification
Inode Modified	Time of last inode modification



Simultaneous Access (Unix)

- i-node
- File table
- File descriptor table

Linux Virtual File System

- Uniform file system interface to user processes
- Represents any conceivable file system's general feature and behavior
- Assumes files are objects that share basic properties regardless of the target file system

Linux Virtual File System

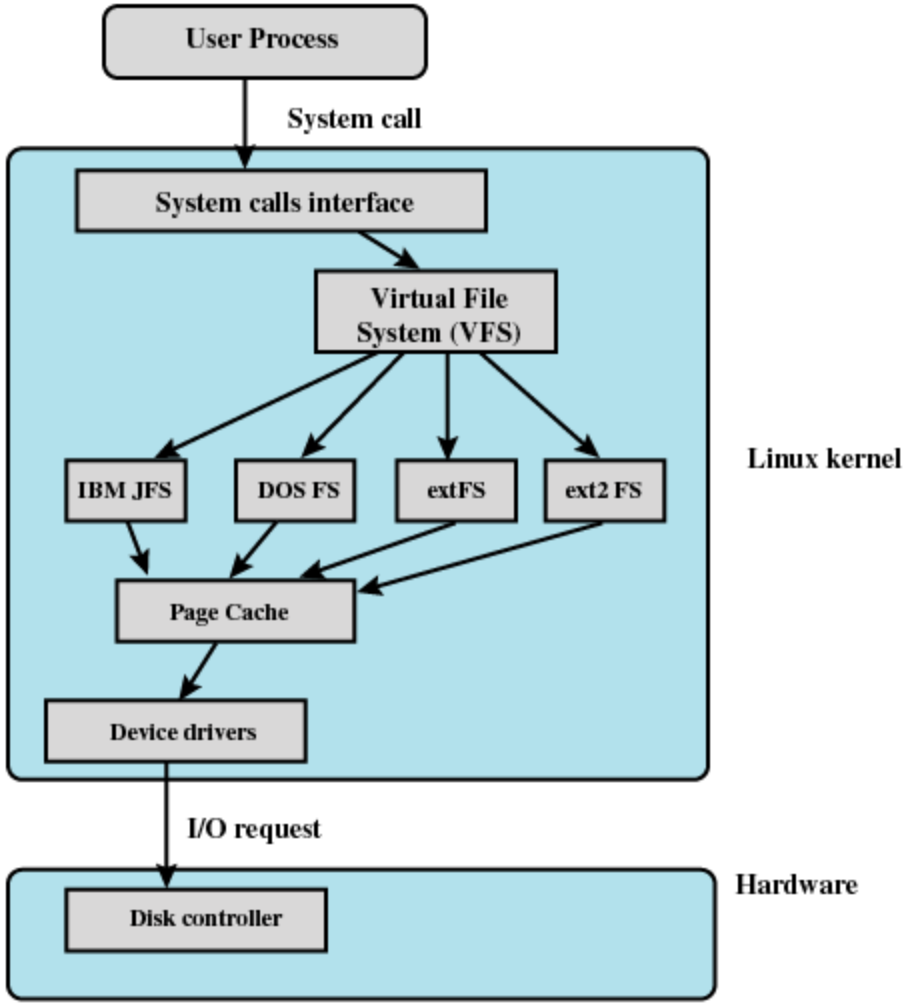


Figure 12.15 Linux Virtual File System Context

Linux Virtual File System

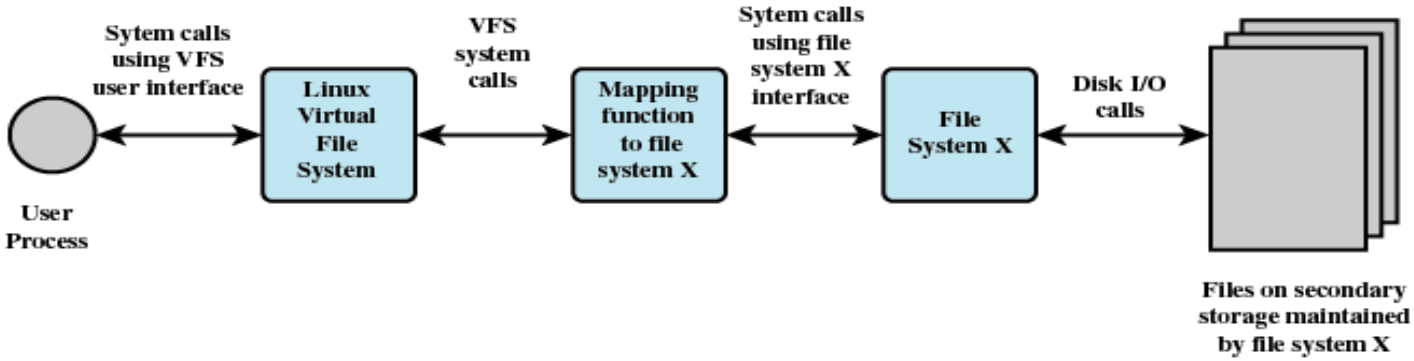
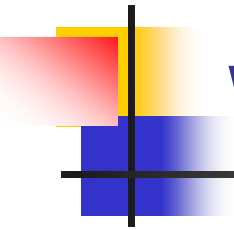


Figure 12.16 Linux Virtual File System Concept

Primary Objects in VFS

- Superblock object
  - Represents a specific mounted file system
- Inode object
  - Represents a specific file
- Dentry object
  - Represents a specific directory entry
- File object
  - Represents an open file associated with a process



Windows File System

- Key features of NTFS
  - Recoverability
  - Security
  - Large disks and large files
  - Multiple data streams
  - General indexing facility

NTFS Volume and File Structure

- Sector
  - The smallest physical storage unit on the disk
- Cluster
  - One or more contiguous sectors
- Volume
  - Logical partition on a disk

Windows NTFS Components

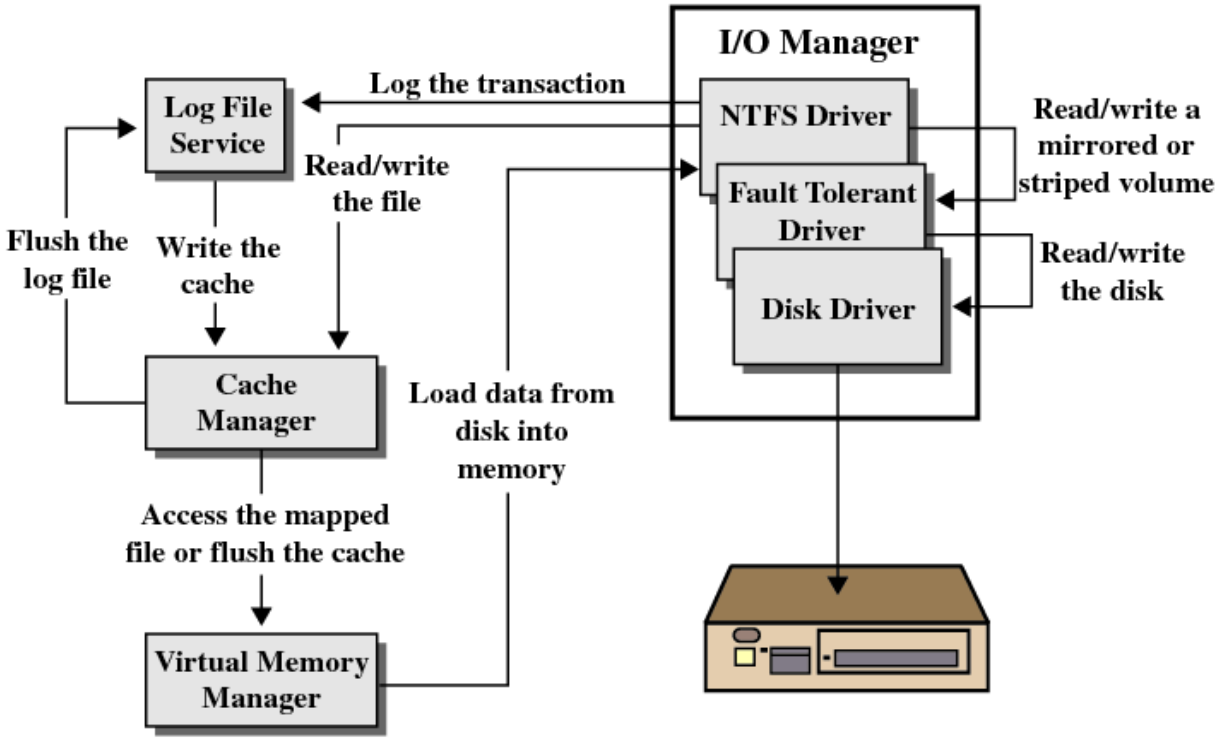


Figure 12.18 Windows NTFS Components